Data 100, Summer 2026

Discussion #7 Solutions

Note: Your TA will probably not cover all the problems. This is totally fine; the discussion worksheets are not designed to be finished within an hour. They are deliberately made slightly longer so they can serve as resources you can use to practice, reinforce, and build upon concepts discussed in lectures, labs, and homework.

Learning Objectives

Gradient Descent

Gradient Descent Fundamentals: Define a gradient and describe how it indicates the direction of steepest ascent or descent. Explain the goal and update rule of gradient descent and the role of the learning rate α. Discuss convergence and convexity and how they relate to finding global minima.

Types of Gradient Descent: Define an epoch. Contrast batch, stochastic, and mini-batch gradient descent. Explain the trade-offs between smaller and larger batch sizes.

Disclaimer: These learning objectives are meant to guide your studying. This list is not exhaustive of all topics covered in lecture or discussion.

Gradient Descent Update Rule:

$$\theta^{(t+1)} = \theta^{(t)} - \alpha \nabla_\theta L(\theta^{(t)})$$

where $\theta^{(t)}$ is the current parameter vector, α is the learning rate (step size), and $\nabla_{\vec{\theta}} L$ is the gradient of the loss function evaluated at $\vec{\theta}^{(t)}$. The negative gradient $-\nabla_\theta L$ always points in the downhill direction of the loss surface.

(Batch) Gradient Descent: GD computes the *true* descent and always descends towards the true minimum of the loss based on a batch (a full dataset). While accurate, it can often be computationally expensive.

(Mini-Batch) Stochastic Gradient Descent: SGD *approximates* the true gradient descent based off of *mini-batch* (a smaller, randomized sample of the dataset). It may not descend towards the true minimum with each update, but it's often less computationally expensive than batch gradient descent.

Epoch: One complete pass through the entire dataset. In SGD, the data is shuffled and divided into mini-batches; after processing all mini-batches (so every datapoint has been used once), one epoch is complete.

Graduate Descent

1. Mir wants to build a model to predict the probability that he will get into each graduate school he applies to. He comes up with the following loss function with the corresponding gradient vector:

$$L(\theta_0, \theta_1) = \frac{1}{n}\sum_{i=1}^{n}\left(y_i - \frac{\theta_0}{x_i} - \ln(\theta_1)\, x_i + \theta_0\theta_1\left(x_i^2 + 1\right)\right)$$

$$\nabla_\theta L = \begin{bmatrix} \frac{1}{n}\sum_{i=1}^{n}\left(-\frac{1}{x_i} + \theta_1 x_i^2 + \theta_1\right) \\ \frac{1}{n}\sum_{i=1}^{n}\left(-\frac{x_i}{\theta_1} + \theta_0 x_i^2 + \theta_0\right) \end{bmatrix}$$

   (a) Mir runs a stochastic gradient descent algorithm. He initializes the model's weights as $\theta_0^{(0)} = 4$ and $\theta_1^{(0)} = 1$, then randomly selects the data point $x_i = 2$ to perform one stochastic gradient descent update. After running one iteration, Mir updates θ_1 to be $\theta_1^{(1)} = -5$. What was Mir's learning rate (α) for this update?

> **Solution:** Based on our gradient from above, we can set up our gradient descent update rule for θ_1 as shown below. Note that we do not need to worry about the summation because we only use one data point in stochastic gradient descent.
>
> $$\theta_1^{(t)} = \theta_1^{(t-1)} - \alpha\left(-\frac{x_i}{\theta_1^{(t-1)}} + \theta_0^{(t-1)}x_i^2 + \theta_0^{(t-1)}\right)$$
>
> Plugging in the values we're given:
>
> $$-5 = 1 - \alpha\left(-\frac{2}{1} + 4\left(2^2\right) + 4\right)$$
>
> This simplifies to
>
> $$-5 = 1 - \alpha(18)$$
>
> From here, we can use some algebra to solve for $\alpha = \frac{1}{3}$.

   (b) Which of the following statements are true? **Select all that apply.**

- ☐ A. The initial weight(s) chosen for gradient descent can impact whether it converges to the true optimal weights.
- ☐ B. The learning rate α can impact whether gradient descent converges to the true optimal weights.
- ☐ C. The choice of loss function can impact whether gradient descent converges to the true optimal weights.

☐ D. Both batch and stochastic gradient descent compute the true gradient of the loss surface during each iteration.

Solution: Option A is correct. If the loss function has local minima other than the global minima, gradient descent may converge to the local minima depending on where the weights are initialized.

Option B is correct. An inappropriate choice of learning rate can cause gradient descent never to converge or potentially even diverge.

Option C is correct. Like Option A, a non-convex loss function may lead to gradient descent converging to the wrong place.

Option D is incorrect. Batch gradient descent computes the true gradient, but stochastic only uses a small sample to compute this gradient.

(c) Mir decides to use stochastic (mini-batch) gradient descent on a dataset with 500 data points. He records that a total of 125 gradients were calculated after 5 epochs. How many data points were in each mini-batch?

Solution:
$\frac{125}{5} = 25$ gradients per epoch
$\frac{500}{25} = 20$ points per mini-batch

Making My Way Downtown

2. Sardaana is a taxi driver shuttling passengers across the city of Berkeley. She decides to build a model to predict the time of each trip, y_i, as a function of the level of traffic, x_i. Her model contains two parameters: θ_0 and θ_1.

   Sardaana's choice of loss function is given below. She decides to use **batch gradient descent** to select the optimal values of θ.

   $$L(\theta_0, \theta_1) = \frac{1}{n}\sum_{i=1}^{n}(y_i - \theta_0 x_i^2 - \theta_1 e^{x_i} + \theta_0\theta_1)$$

   (a) What is the gradient vector, $\nabla_\theta L$, for Sardaana's choice of loss function? Show your work in the box below.

   Solution: To compute the gradient vector, $\nabla_\theta L = \begin{bmatrix} L\theta_0 & L\theta_1 \end{bmatrix}^T$, we first take the derivative of the loss function with respect to θ_0. All terms not involving θ_0 are ignored.

   $$L\theta_0 = \frac{1}{n}\sum_{i=1}^{n}(-x_i^2 + \theta_1)$$

   Then, we take the derivative of the loss function with respect to θ_1.

   $$L\theta_1 = \frac{1}{n}\sum_{i=1}^{n}(-e^{x_i} + \theta_0)$$

   Stacking these derivatives in a vector gives us $\nabla_\theta L$.

   (b) Which of the following are **not** appropriate choices for the learning rate, α, in gradient descent?

   ☐ A. $\alpha = -0.5$
   ☐ B. $\alpha = -0.2$
   ☐ C. $\alpha = 0$
   ☐ D. $\alpha = 0.2$
   ☐ E. $\alpha = 0.5$

   Solution: Recall the gradient descent update rule: $\theta^{(t+1)} = \theta^{(t)} - \alpha\nabla_\theta L(\theta)$ As shown in lecture, this rule "shifts" each subsequent guess for θ in the opposite

direction to the gradient of the loss surface at time t. The negative of the gradient vector, $-\nabla_\theta L(\theta)$, always points towards the minimum of the loss surface, i.e. $-\alpha\nabla_\theta L(\theta)$ must always be negative in value: otherwise, the next guess for θ will take us higher up the loss surface and result in greater loss. Then $\alpha = -1$ and $\alpha = -0.5$ cannot be appropriate choices. We also cannot have $\alpha = 0$ because this would mean that the guess for θ never updates beyond the initial guess $\theta^{(0)}$.

Loss Function Design

3. We are given the following loss function:

$$L(\theta, \vec{x}, \vec{y}) = \frac{1}{n} \sum_{i=1}^{n} \left(\theta^2 x_i - \log(y_i) \right)$$

(a) Explicitly write out the update equation for $\theta^{(t+1)}$ in terms of x_i, y_i, $\theta^{(t)}$, and α. Simplify your expression by plugging in the constant learning rate $\alpha = 0.5$.

Solution:

$$\theta^{(t+1)} \leftarrow \theta^{(t)} - \alpha \frac{\partial L}{\partial \theta}\bigg|_{\theta=\theta^{(t)}}$$

$$\frac{\partial L}{\partial \theta} = \frac{1}{n} \sum_{i=1}^{n} 2\theta x_i$$

Given $\alpha = 0.5$:

$$\theta^{(t+1)} = \theta^{(t)} - \frac{1}{2n} \sum_{i=1}^{n} 2\theta^{(t)} x_i = \theta^{(t)} \left(1 - \frac{1}{n} \sum_{i=1}^{n} x_i \right) = \theta^{(t)}(1 - \bar{x})$$

(b) Rohan investigates further, and it turns out that despite experimenting with learning rates, the gradient descent algorithm still does not converge for his dataset! Explain why this happens by reasoning about the loss function that we are optimizing.

Solution: The loss function is broken. We can often make it move towards negative infinity by pushing the magnitude of θ to be larger and larger with the correct sign. This depends on what $\bar{x}$ is.

(c) As $t \to \infty$, what are the required conditions for $\theta^{(t)}$ to converge with the learning rate in part (a)? To what can it converge?

Solution: Simplifying the recurrent relation, we get:

$$\theta^{(t)} = (1 - \bar{x})^t \theta^{(0)}$$

For this quantity to not explode as $t \to \infty$, we need $0 \leq \bar{x} < 2$. Under those conditions, it will either converge to 0 or $\theta^{(0)}$.
With negative values of $\bar{x}$, $\theta^{(t)}$ will be multiplied by a value greater than 1, and infinitely increase. With values greater than or equal to 2, $\theta^{(t)}$ will be multiplied by a negative value less than -1, and will oscillate forever.

Infinite Descent

4. Emrie used gradient descent to minimize the function L(θ) with respect to the one dimensional parameter θ. Unfortunately, he forgot to save some of the initializations he used in Jupyter Notebook (always save!). $L(\theta)$ is defined below.

$$L(\theta) = \begin{cases} 2\theta^2 + 4 & \theta \leq 0 \\ \frac{\theta}{2} + 4 & \theta > 0 \end{cases}$$

Assume the derivative at $\theta = 0$ with respect to θ is 0.

Suppose Emrie used a learning rate of 0.5, but he cannot remember the initialization of θ, where $\theta^{(0)} > 0$. He observed the behavior that $\theta^{(i)}$, for any update i, oscillated between two unique values $\theta^{(0)}$ (initialization) and $\theta^{(1)}$ (after one gradient step), causing gradient descent to not converge to the global minimum.

(a) If gradient descent oscillates between two same unique values (in other words, two and only those two values), which of the following choices must be true?

☐ A. The sign of the gradient oscillates.

☐ B. The sign of the gradient does not oscillate.

☐ C. The magnitude of the gradient oscillates.

☐ D. The magnitude of the gradient does not oscillate.

Solution: A standard gradient update is as follows:

$$\theta^{(i+1)} = \theta^{(i)} - \alpha \frac{dL}{d\theta}$$

Given that $\alpha > 0$, the only way for an update to move between two unique points back and forth is for the sign of the gradient to alternate every update. However, if the magnitude oscillated, it would end up at a unique point given that the learning rate is fixed. Hence, the magnitude is constant but the sign is oscillating.

(b) Following from the same context as the previous subpart, what was Emrie's initialization of $\theta^{(0)}$ that caused the behavior in the previous subpart?

Solution: We know that the sign of the gradient must oscillate, but the magnitude must not, so θ must oscillate between positive and negative values. We also know that for some two values of θ, their gradient magnitudes are equal. This gradient magnitude must be $\frac{1}{2}$ since the right side of the piecewise function has a constant gradient for $\theta > 0$:

$$\frac{\partial L}{\partial \theta} = \frac{1}{2} \quad (\theta > 0)$$

Hence, the gradient at the θ value on the left side of the piecewise function must be $-\frac{1}{2}$.

$$\frac{\partial L}{\partial \theta} = \boxed{4\theta = -\frac{1}{2}} \quad (\theta \leq 0)$$

yielding

$$\theta^{(1)} = -\frac{1}{8}$$

Then, we can compute the correct $\theta^{(0)}$ by "reversing" the gradient step and adding $\alpha \frac{dL}{d\theta} = \frac{1}{4}$:

$$\theta^{(1)} = \theta^{(0)} - \alpha \frac{dL}{d\theta} \quad \Longleftrightarrow \quad \theta^{(0)} = \theta^{(1)} + \alpha \frac{dL}{d\theta} - \frac{1}{8} + \frac{1}{4} = \frac{1}{8}$$

(c) Suppose Emrie used a fixed learning rate with gradient descent, but he forgot all the initializations and other configurations. What are the possible resulting outcomes?

Solution: We could oscillate infinitely between two or more points or converge to the global minimum in a finite number of steps depending on the learning rate and initialization.